

Project Specification for Coursework (without Code)

Motivations

If you take all lab sessions as a project, this document could be a project specification for the coursework. Usually, once a customer (in INT104, your teacher) makes a series of requirements (as you have seen in the CW task sheets), the engineer (you, the student) should write a project specification that specifies the technical requirements and proposed solutions for a project. The document could be considered as a project specification for a demonstration.

For INT104, the mini project (your coursework) is too small for writing a formal project specification, so if you did not attempt to write a project, you need not bother to write one.

By showing a possible way for the project to be specified, the student who cannot finish the coursework could reuse the code and solution mentioned in the document. However, this is only one possible way to interpret the requirements in the coursework task sheet. ***The student can analyse the requirements in any other sensible way if such interpretation is aligned with the requirements in the task sheets.***

General Requirements

In this module, three task sheets are provided. The task sheets, "Unsupervised Learning Exercise" and "Supervised Learning Exercise", can be taken as instructions for lab sessions. The Python script (source code) should be submitted. To ensure it is the student who writes the code, a live demonstration session is required, where the student needs to run the code submitted on LMO with a new dataset provided.

In both "Unsupervised Learning Exercise" and "Supervised Learning Exercise", the requirement in task 1 requires algorithms (both supervised and unsupervised) to be run with multiple configurations and multiple sets of features. However, in task 2, ***the live demonstration does not require the implementation with multiple configurations and multiple sets of features***, but requires a better dedicated metric.

This suggests that during the lab session, the student should pay attention to ***the combination of features and configurations that produce the best-performing system***, which has a higher potential to deliver a better result in the live demonstration sessions. Moreover, the way the code is written should ***consider the ease of reusing part of the code*** for training over the live demonstration session, which potentially requires self-defined classes and functions, or the use of loop structures.

Comparing the task sheet of "lab report" with the other task sheets, the requirement for the lab report falls short of the requirement in unsupervised/supervised learning exercises. ***This allows only selected results to be documented in the lab report.*** As a result, during the lab sessions, the

student should consider which part of the experiment results should be included in the experiment report to produce a better lab report.

Given the general requirements, the remaining parts of the specification will be organised as: the unsupervised learning and supervised learning exercises will be discussed first then followed by the possible organisation of lab reports.

Unsupervised Learning

The key requirements of task 1 in unsupervised learning exercise are:

- With **at least three sets of features**, divide the provided dataset into **four clusters**. For each set of features, **at least three clustering processes should be applied**.

More specifically,

- Apply transforms to raw data and obtain three sets of features;
- Implement three clustering models: GMM, hierarchical clustering (HC) and k-means (KM).
- An attempt to try different settings for each clustering method
- A table showing the best-performed clustering results **for each set of features and each clustering method**

From the requirement, a proposed solution of the exercise could be:

1. Define three sets of features for the dataset
2. With each set of features, apply GMM, hierarchical clustering and k-means and observe the performance of the clustering algorithms by comparing the ratio between the average intra-cluster distances between samples and the average inter-cluster distances between samples
 - For each set of features, comparing the performance of different clustering algorithms
3. For each clustering algorithm, change at least one setting and observe how the performance of the clustering algorithm changes
 - For each clustering method, compare the performance of different sets of features

As trying the same clustering method with different features is a highly repetitive process, it is better to write a self-defined function that takes a specific input and a specific setting for each clustering process. The output of such functions should be the dedicated metric, in this case, the ratio between the average intra-cluster distances and the average inter-cluster distances.

As a result, there should be a function that compute the ratio between the average intra-cluster distances and the average inter-cluster distances, e.g.,

▼ Metric for Clustering

Python

```
import numpy as np
from sklearn.metrics import pairwise_distances

def distance_ratio(feature_matrix, labels):
    """
```

Computes the ratio between the average intra-cluster distance and the average inter-cluster distance.

Parameters:

`feature_matrix` (numpy.ndarray): A 2D array where rows represent samples and columns represent features.

`labels` (numpy.ndarray): An array of cluster labels for each sample.

Returns:

float: The ratio of average intra-cluster distance to average inter-cluster distance.

```
"""
```

```
# Calculate pairwise distances between all samples
```

```
# Get unique cluster labels
```

```
# Initialize variables to compute intra-cluster and inter-cluster distances
```

```
# Calculate intra-cluster distances
```

```
# Calculate inter-cluster distances
```

```
# Calculate average distances
```

```
# Calculate the ratio
```

```
return ratio
```

Start with k-means algorithm, by reading the API documentation, the most straightforward and interesting setting is to investigate the performance improvement with the algorithm running for multiple times (i.e. 'n_init'), which examines the impacts of initial values for training. As a result, the function should receive a setting that commands the specific value for 'n_init'. Moreover, **the label of clusters for each sample should be the output of the function to enable the computation of performance metric over the live demonstration session.** So the function can be written as:

▼ Clustering with k-means

Python

```
from sklearn.cluster import KMeans
```

```
def kmeans_clustering(feature_matrix, n_attempt='auto'):
```

```
    """
```

```
    Clusters the input feature matrix into n_clusters using K-means.
```

Parameters:

`feature_matrix` (numpy.ndarray): A 2D array where rows represent samples and columns represent features.

`n_attempt` (int): The number of attempts for initial values.

Returns:

ratio: The ratio of average intra-cluster distance to average inter-cluster distance.

`labels` (numpy.ndarray): An array of cluster labels for each sample.

```
    """
```

```

# Initialize the KMeans model

# Fit the model to the feature matrix

# Get the labels for each sample

# Compute the metric
ratio = distance_ratio(feature_matrix, labels)

return ratio, labels

```

Now, for similar reason, another function is written, which also enables an input for a feature matrix and a value of 'n_init'.

▼ Clustering with GMM Python

```

from sklearn.mixture import GaussianMixture

def gmm_clustering(feature_matrix, n_attempt='auto'):
    """
    Clusters the input feature matrix into n_clusters using Gaussian Mixture Model (GMM).

    Parameters:
    feature_matrix (numpy.ndarray): A 2D array where rows represent samples and columns represent features.
    n_attempt (int): The number of attempts for initial values.

    Returns:
    ratio: The ratio of average intra-cluster distance to average inter-cluster distance.
    labels (numpy.ndarray): An array of cluster labels for each sample.
    """
    # Initialize the Gaussian Mixture Model

    # Fit the model to the feature matrix

    # Get the labels for each sample

    # Compute the metric
    ratio = distance_ratio(feature_matrix, labels)

    return ratio, labels

```

Finally, the function of hierarchical clustering may investigate the way that inter-cluster distances being calculated.

▼ Clustering with Hierarchical Clustering Python

```

from scipy.cluster.hierarchy import linkage, fcluster

def hierarchical_clustering(feature_matrix, fc_method='ward'):
    """
    Clusters the input feature matrix into n_clusters using hierarchical clustering.

```

```

Parameters:
    feature_matrix (numpy.ndarray): A 2D array where rows represent samples and columns represent features.
    fc_method (string): The method of calculating the distance between each pair of clusters.

Returns:
    ratio: The ratio of average intra-cluster distance to average inter-cluster distance.
    labels (numpy.ndarray): An array of cluster labels for each sample.
"""

# Perform hierarchical clustering

# Assign cluster labels

# Compute the metric
ratio = distance_ratio(feature_matrix, labels)

return ratio, labels

```

Now, in a new Python script you could write the code for your experiments. Firstly, we need to load the data and standardise the data, e.g.

▼ Import Data and Standardisation Python

```

import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler, Normalizer

# Load Data
[.....]

# Scale Data
[.....]

```

Now, k-means, GMM and hierarchical clustering are applied to three sets of features.

▼ Comparing Different Features Python

```

from cluster_functions import kmeans_clustering, gmm_clustering, hierarchical_clustering

# Initialise Result Matrix

# Initialise Data

# Experiment on

# Store results in a dataframe

# Set display format
pd.set_option('display.float_format', '{: .2f}'.format)

```

```
# Display Results
print(df)
```

The result can be displayed as:

▼ Results for Feature Comparison

Markdown

	k-means	GMM	Hierarchical
Standard	0.32	0.37	0.32
Range	0.30	0.35	0.30
Normalise	0.16	0.24	0.16

The student is expected to draw conclusions with the obtained result and discuss the possible reason that leads to the result.

As the normalised feature gives the best result in the previous result, the next experiment could be performed with the best features, e.g.

▼ Comparing Different Settings in Clustering

Python

```
# The second experiment
[.....]
```

```
print(f'for k-means, n_init=1: {ratio1_km:.2f}, n_init=10: {ratio2_km:.2f}')
print(f'for GMM, n_init=1: {ratio1_gmm:.2f}, n_init=10: {ratio2_gmm:.2f}')
print(f'for Hierarchical Clustering, ward: {ratio1_hc:.2f}, single: {ratio2_hc:.2f}')
```

The result is displayed as:

▼ Results of Setting Comparison

Markdown

```
for k-means, n_init=1: 0.16, n_init=10: 0.16
for GMM, n_init=1: 0.24, n_init=10: 0.39
for Hierarchical Clustering, ward: 0.16, single: 0.10
```

The setting with a better result should be attempted first in the live demonstration session, i.e.

- `n_init = 1` for k-means
- `n_init = 1` for GMM
- `fc_method = 'single'` for hierarchical clustering

Supervised Learning

According to the task sheet, the key requirements for supervised learning exercise are:

- Define three sets of features
- Train three different classifiers
- Show the best-performed models
- Train an ensemble model
- Apply cross validation

As three sets of features are used to train three classifiers, so **the features used for training different classifiers can be different across different classifiers.** Similar with the case of

unsupervised learning, to use different configurations over the training process, a sets of Python functions should be defined.

However, given the consideration of applying cross validation and training ensemble models, the self-defined Python functions should return the resulting classifiers instead of a specific metric. As the task sheet requires comparing different settings of classifiers, the self-defined functions should provide an input as the setting of classifier. Moreover, it would be great to show the accuracy of the training dataset for reference.

For the Naive Bayes, the prior probability distribution will be investigated.

▼ Function of Training a Naive Bayes Classifier

Python

```
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB

def train_naive_bayes_classifier(features, label, priors=None):
    """
    Trains a Gaussian Naive Bayes classifier on the provided feature matrix.

    Parameters:
    features (numpy.ndarray): Feature matrix (2D array).
    label (numpy.ndarray): Labels (1D array).
    priors (numpy.ndarray): Prior probabilities (1D array).

    Returns:
    model: Trained Naive Bayes classifier.
    """

    # Create a Gaussian Naive Bayes classifier

    # Train the model

    # Calculate accuracy of the training dataset
    [.....]

    return model
```

For decision tree, the criterion of splitting nodes is examined.

▼ Function of Training a Decision Tree Classifier

Python

```
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier

def train_decision_tree_classifier(features, label, dt_criterion='gini'):
    """
    Trains a Decision Tree classifier on the provided feature matrix.

    Parameters:
    features (numpy.ndarray): Feature matrix (2D array).
    label (numpy.ndarray): Labels (1D array).
```

```
dt_criterion (string): 'gini', 'entropy'
```

```
Returns:
```

```
model: Trained Decision Tree classifier.
```

```
"""
```

```
# Create a Decision Tree classifier
```

```
# Train the model
```

```
# Calculate accuracy of the training dataset
```

```
return model
```

For KNN, it is very obvious that the number of neighbours will be investigated.

▼ Function of Training a kNN Classifier

Python

```
import numpy as np
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
def train_knn_classifier(features, label, n_neighbors=3):
```

```
    """
```

```
    Trains a K-Nearest Neighbors classifier on the provided feature matrix.
```

```
    Parameters:
```

```
    features (numpy.ndarray): Feature matrix (2D array).
```

```
    label (numpy.ndarray): Labels (1D array).
```

```
    n_neighbors (int): Number of neighbors to use for KNN.
```

```
    Returns:
```

```
    model: Trained KNN classifier.
```

```
    """
```

```
# Create a KNN classifier
```

```
# Train the model
```

```
# Calculate accuracy of the training dataset
```

```
[.....]
```

```
return model
```

Finally, a Python function that trains a voting based ensemble classifier is written. The voting method in the classifier will be investigated. However, other means of ensemble classifiers, such as random forest, could be used with investigation on specific settings investigated.

▼ Function of Training an Ensemble Classifier

Python

```
from sklearn.ensemble import VotingClassifier
```

```
def train_ensemble_classifier(features, label, knn, nb, dt, ec_voting='hard'):
```

```
    """
```

```
    Trains an ensemble classifier combining KNN, Naive Bayes, and Decision Tree classi
```

```
fiers.

Parameters:
features (numpy.ndarray): Feature matrix (2D array).
label (numpy.ndarray): Labels (1D array).
knn: kNN classifier.
nb: Naïve Bayes classifier.
dt: Decision Tree classifier.
ec_voting: 'hard' or 'soft'.

Returns:
model: Trained ensemble classifier.
"""

# Create an ensemble classifier using majority voting

# Train the ensemble model

# Calculate accuracy of the training dataset
```

With these self-defined functions, another Python script can be written to perform the dedicated experiments. Firstly, the data is loaded. Unlike the experiments in the unsupervised learning exercise, the label information should be loaded this time. Moreover, besides transform the raw feature into featured matrices, the feature matrices should also be splitted into two sections: the training data and the testing data.

▼ Load data and split the training and test dataset Python

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, Normalizer

# Load Data

# Scale Data

# Split Training Data and Testing Data
```

Next, we could train the required classifiers with different settings.

▼ Experiments for simple classifier Python

```
from sklearn.metrics import accuracy_score
from classifier_functions import train_knn_classifier, train_naive_bayes_classifier, train_decision_tree_classifier

# kNN experiments

# Decision tree experiments

# Naive Bayes experiments
```

```
# Show results of single classifier
```

The result could be shown as:

▼ Output

Python

```
kNN has achieved an accuracy of 0.63 for the training dataset.
kNN has achieved an accuracy of 0.60 for the training dataset.
Decision Tree has achieved an accuracy of 0.93 for the training dataset.
Decision Tree has achieved an accuracy of 0.93 for the training dataset.
Naive Bayes Classifier has achieved an accuracy of 0.16 for the training dataset.
Naive Bayes Classifier has achieved an accuracy of 0.35 for the training dataset.
for kNN, k=3 accuracy: 0.41, k=5 accuracy: 0.42
for decision tree, gini accuracy: 0.34, entropy accuracy: 0.37
for naive Bayes, random accuracy: 0.14, prior accuracy: 0.33
```

Next, we could try different setting and train ensemble classifiers:

▼ Train ensemble classifiers

Python

```
from classifier_functions import train_ensemble_classifier

# Ensemble Classifier Experiments
[.....]
```

The output in command line:

▼ Output

Python

```
Ensemble Model has achieved an accuracy of 0.67 for the training dataset.
Ensemble Model has achieved an accuracy of 0.90 for the training dataset.
for ensemble learning, hard accuracy: 0.44, soft accuracy: 0.39
```

Though the experiment is finished, the results should be analysed and selected. The student should be aware that not all experiment results would serve as an evidence / a means of support to the proposed conclusion in the report hence the presentation of experiment should be selected.

Lab Report

The student could address their findings in the lab report using the experiment results obtained. Moreover, the required experiments for the lab report fall short of the other coursework ('unsupervised learning exercise' and 'supervised learning exercise'). As a result, the student should selectively report the experiment results.

For tasks 1 & 2, three standardisation methods should be applied to the data. The comparison requires one metric. The metric does not necessarily evaluate the ease of classifying programme information (e.g. conditional probability distribution for different programmes). Simple metrics, such as variance, may also work by giving a different conclusion and motivation for feature comparison.

For task 3, three sets of features are required, but only three clustering configurations are needed simultaneously. That is to say, for each set of features, it is possible to try only one clustering

method as long as three configurations of the clustering algorithm are tried. The student is expected to analyse the design of the best-performed clustering system for each set, which is taken as the justification of the experiment design. Finally, the resulting cluster should be compared with programme information, which could be as simple as comparing the labelling information and the clustering labels. However, more complex methods have been provided in the Deepnote Python script.

Similar to task 3, the student may present the best-performed classifier for each set of features in task 4. The feature sets could be the same across all tasks. The design of the ensemble classifier, which may only use the best-performed set of features, requires a comparison between different settings. Moreover, a justification of the selection features and ensemble methods is expected in the report, besides the comments on the final results.

Finally, the student needs to provide a longer comment on the results obtained, which should be supported by experiment results (as evidence for the conclusion). Future work should specify any further work that could be done to address new findings.

As a final remark, this interpretation of the requirement for the lab report is just one of many possible interpretations. The student could interpret the requirement in other ways as long as such interpretations do not interfere with the requirement provided in the task sheet.

Live Demonstration

Setup Live Demonstration Environment

Using our script to configure the environment is optional. If you're already familiar with Python and prefer to set up your environment manually, you can do so directly. This approach may even save you time. If you configure the environment by yourself, you can skip to the next section: "Adaptation to New Dataset (for Unsupervised)"

1. On LMO, download and unzip your submitted coursework file

▼ Coursework



Coursework - Lab Report (Updated on 2nd Apr)

View



Coursework - Unsupervised Learning (Updated on 2nd Apr)

View



Coursework - Unsupervised Learning Exercise Submission

Make a submission

Opened: Tuesday, 15 April 2025, 00:00

Due: Thursday, 1 May 2025, 00:00

李圣辰(Shengchen DR LJ)-8563

2. Then, download all files in the folder of "Live Demonstration Files" on LMO



Live Demonstration Files

View

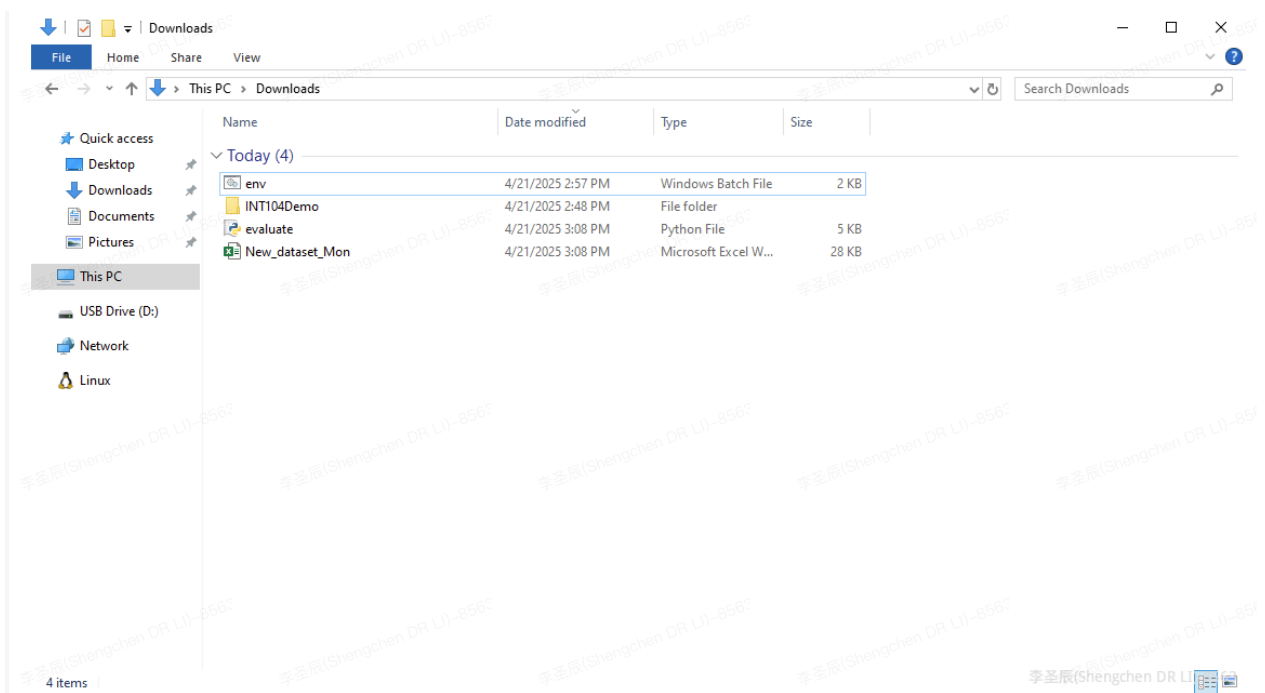


Not available unless:

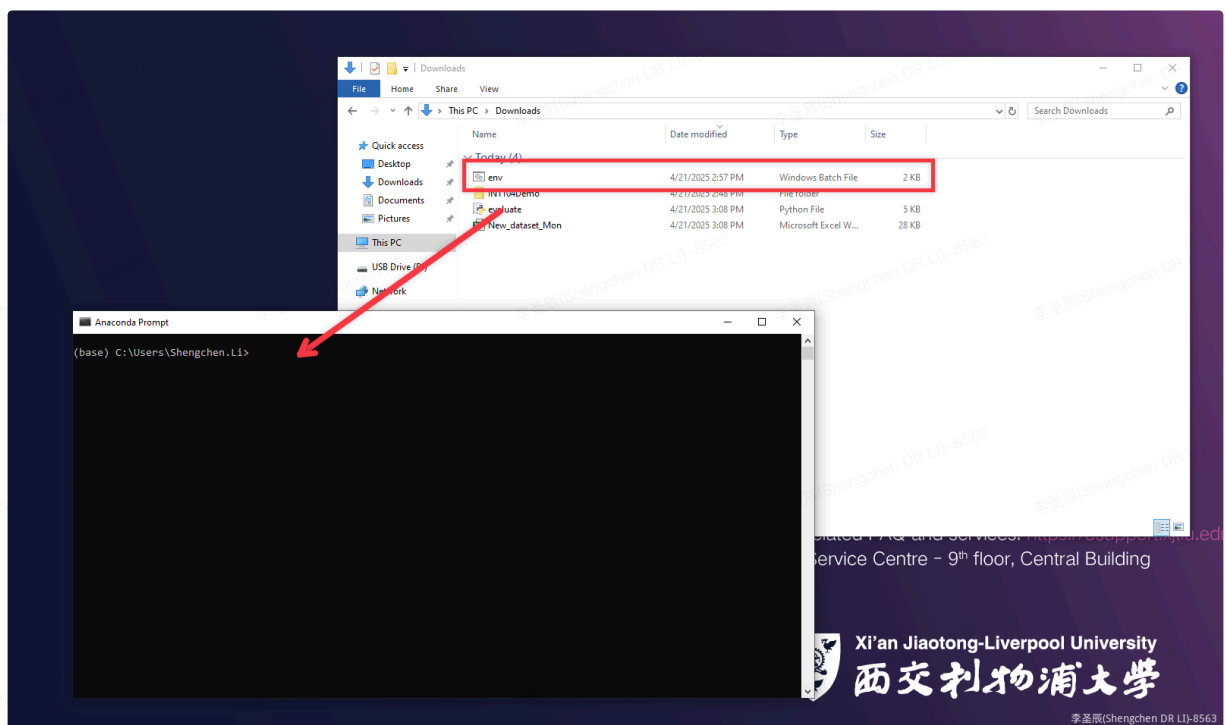
- It is after **5 May 2025, 14:00**
- It is before **5 May 2025, 19:00**

▼ Lecture 1 - AI Systems (Overview)

3. Place all files in a folder on the local computer



4. Open "Anaconda Prompt", then drag the file "env.bat" into "Anaconda Prompt".



5. Then you should see the following interface in "Anaconda Prompt"

```
Anaconda Prompt - C:\Users\Shengchen.Li\Downloads\env.bat

(base) C:\Users\Shengchen.Li>C:\Users\Shengchen.Li\Downloads\env.bat
Creating Conda environment 'int104' with Python 3.9...
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\Shengchen.Li\.conda\envs\int104

added / updated specs:
- python=3.9

The following packages will be downloaded:

package | build | size
----- | ----- | -----
ca-certificates-2025.2.25 | haa95532_0 | 130 KB
openssl-3.0.16 | h3f729d1_0 | 7.8 MB
pip-25.0 | py39haa95532_0 | 2.5 MB
python-3.9.21 | h8205438_1 | 19.6 MB
setuptools-75.8.0 | py39haa95532_0 | 1.7 MB
tzdata-2025a | h04d1e81_0 | 117 KB
vc-14.42 | haa95532_5 | 11 KB
vs2015_runtime-14.42.34433 | hbf602d_5 | 1.2 MB
wheel-0.45.1 | py39haa95532_0 | 145 KB
```

6. Wait until you see the following interface

```
Anaconda Prompt - C:\Users\Shengchen.Li\Downloads\env.bat

Downloading and Extracting Packages:
Preparing transaction: done
Verifying transaction: done
Executing transaction: done

Successfully created environment 'int104' with numpy installed.
To activate the environment, run:
  conda activate int104

Press any key to continue . . .
```

7. Then press a key on your keyboard. Close the prompt window.

```
Anaconda Prompt

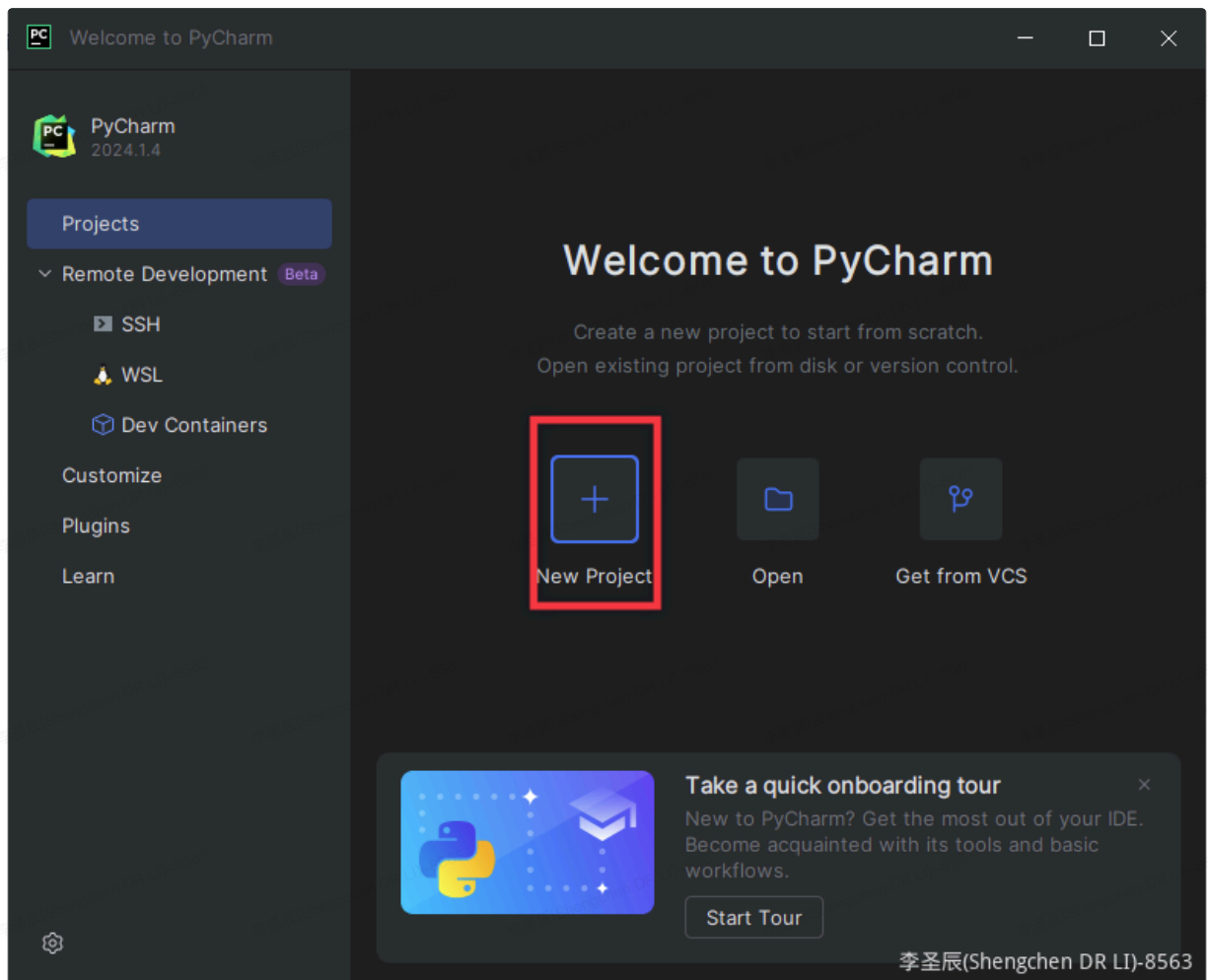
Downloading and Extracting Packages:
Preparing transaction: done
Verifying transaction: done
Executing transaction: done

Successfully created environment 'int104' with numpy installed.
To activate the environment, run:
    conda activate int104

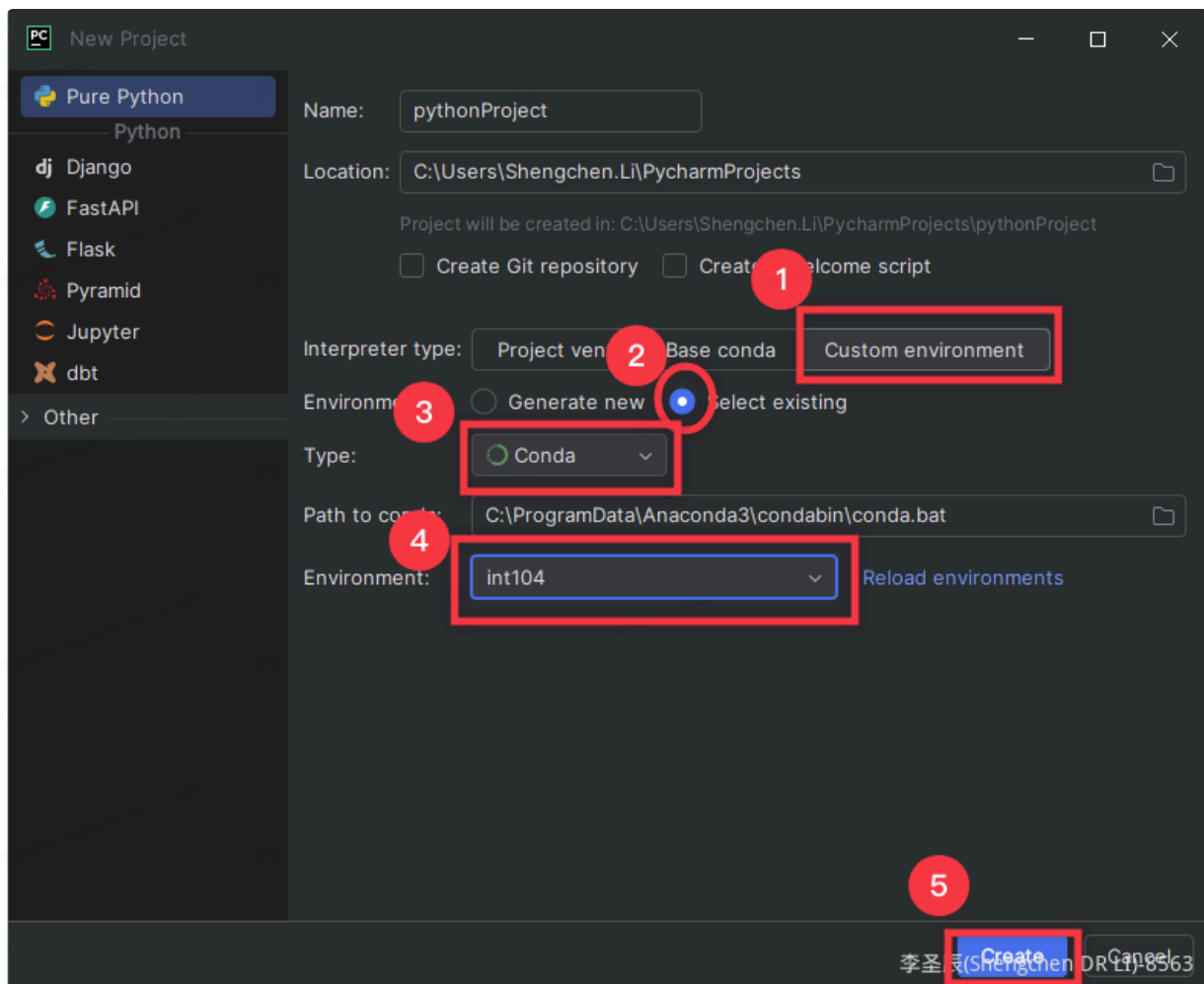
Press any key to continue . . .

(base) C:\Users\Shengchen.Li>
```

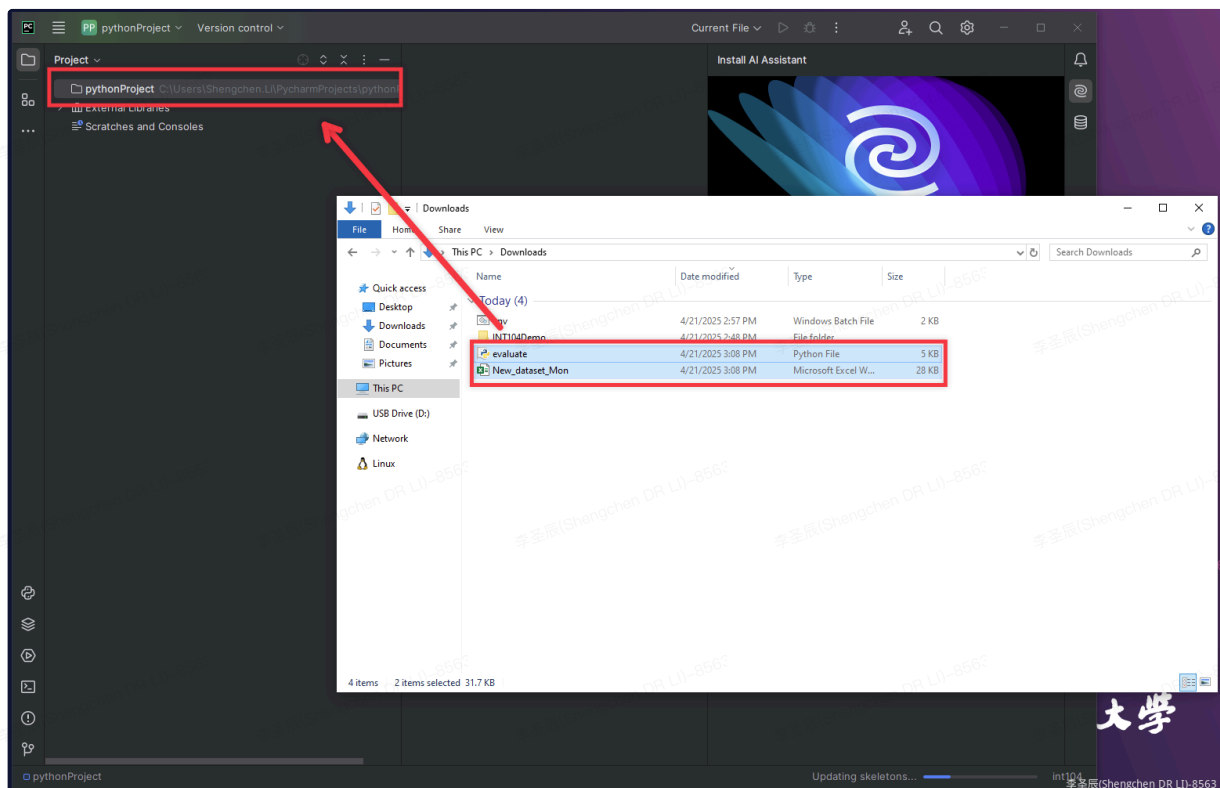
8. Open PyCharm. Click "New Project".



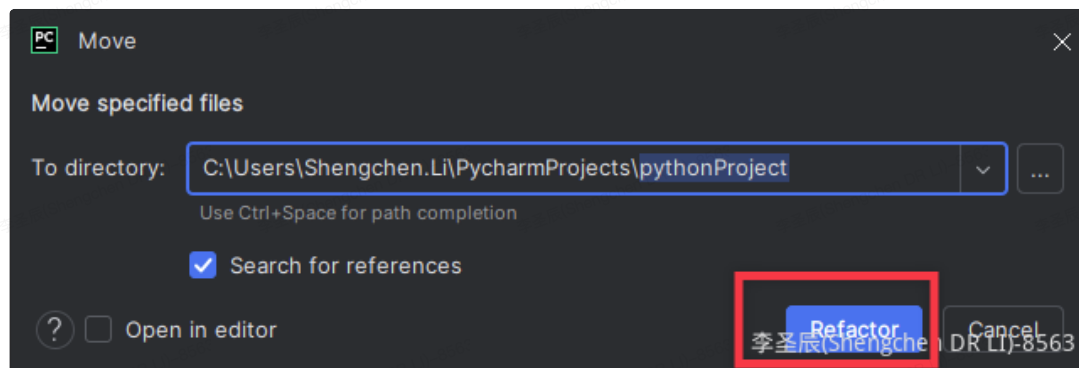
9. Then click "Customize Environment" —> Environment: "Select existing" —> "Type: "Conda" —> Environment: "int104". After than click "Create".



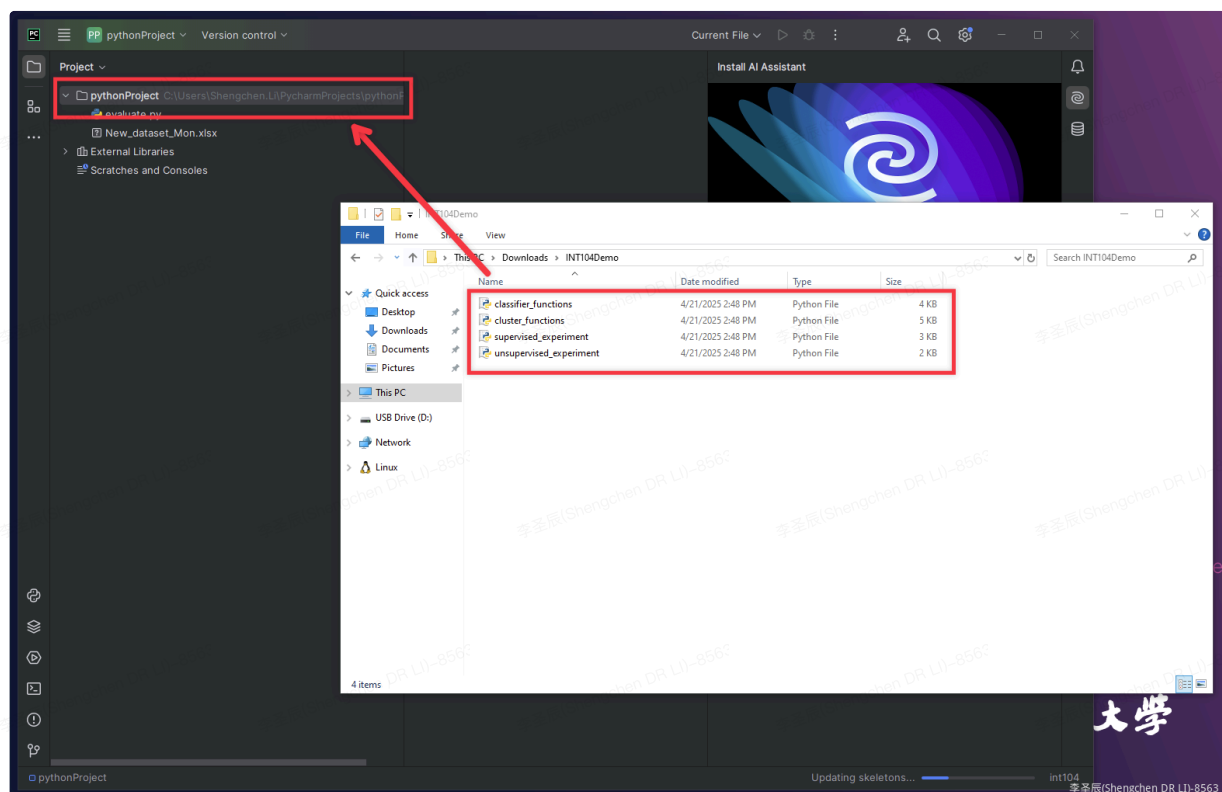
10. Open the folder that the downloaded files are stored (usually "Downloads"), select the data file and evaluate library file ("evaluate.py") and drag the files into Pycharm



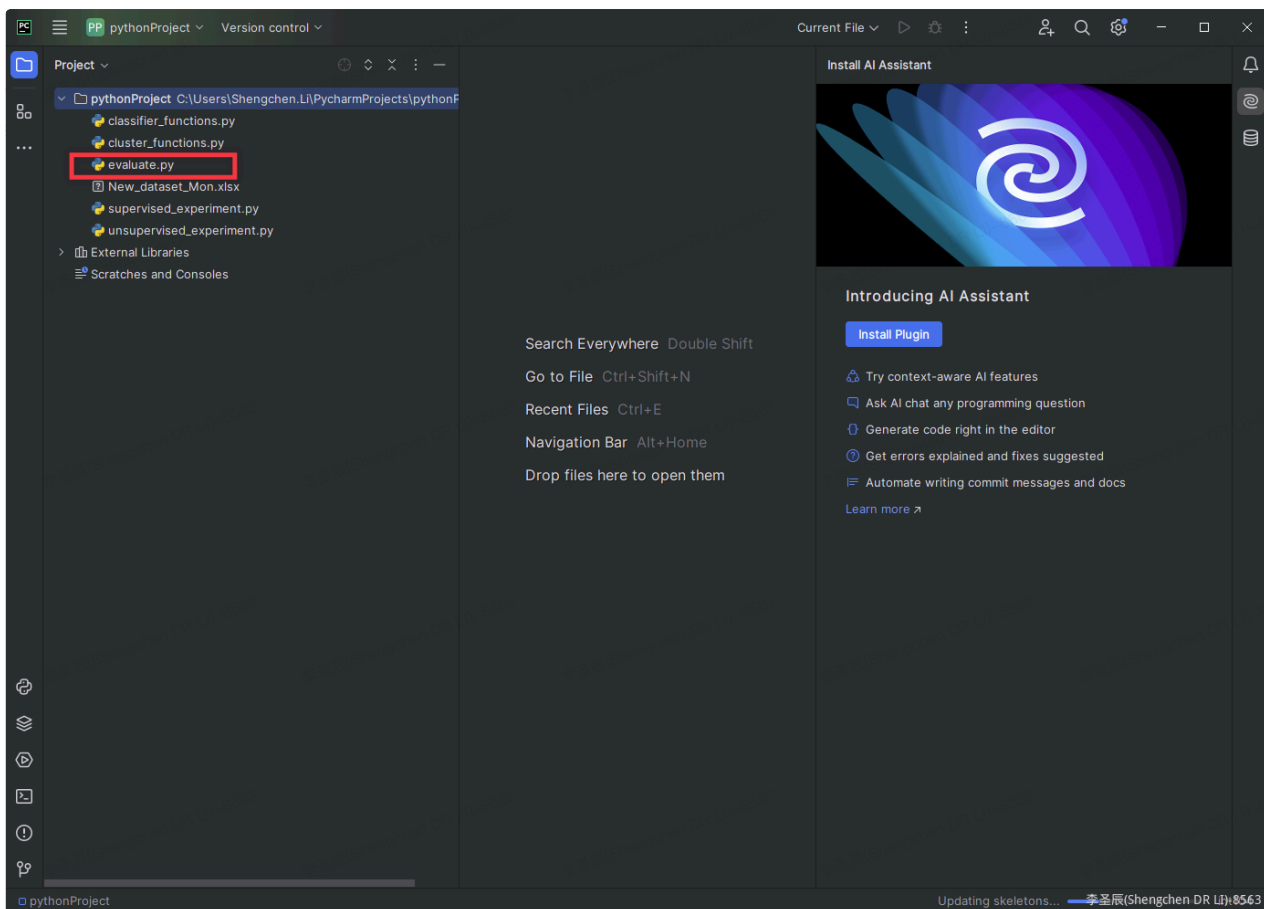
11. In the new window appeared, click "Refactor".



12. Next, drag all Python scripts that you need into PyCharm. Click "Refactor" in the next interface as you done in Step 11.



Now, you can start to change your Python script to adapt the new dataset. Possibly, the interface looks like:



Please make sure your *evaluate.py* and your other code are in the same folder.

Adaptation to New Dataset (for Unsupervised)

1. An Excel file containing the new dataset will be provided on LMO, named as "New_Dataset_xxx.xlsx".
2. At the top of the your Python script, you should add the following command:

```
from evaluation import evaluate_clustering
```

Python

3. Then you can revise your Python script accordingly.
4. After finishing the clustering process, you need to insert the following command (ASSUMING YOU STORE THE LABEL OF CLUSTERING IN "labels", THE ORIGINAL FEATURE MATRIX IS "X")

```
evaluate_clustering(X, labels)
```

Python

5. The the prompt will display the following information:

```
"Execution Time: xx-xx-xx xx:xx:xx - Ratio: 0.xxxx"
```

纯文本



You ONLY need to adapt ONE clustering system for each clustering method (k-means, GMM, hierarchical clustering)

Adaptation to New Dataset (for Supervised Learning)

1. There will be two Excel files containing the new dataset.

- The training data: "Training_Dataset_xxx.xlsx"
- The testing data: "Testing_Dataset_xxx.xlsx"
- There is also a new bin "check.bin" file that is used to encrypt other information



You need to move ALL THREE FILES into Pycharm

2. At the top of the your Python script, you should add the following command:

```
from evaluation import evaluate_classification
```

Python

Please load the data in "Training_Dataset_xxx.xlsx" for training.

3. Then you can revise your Python script accordingly.

4. After finishing the adaptation process,

- a. Load data in "Texting_Dataset_xxx.xlsx" (assuming you stored your data in the variable "valid_data").
- b. Predict the results with the method

```
pred_label = [classifier].predict(valid_data)
```

Python

c. Call the following command

```
evaluate_classification(pred_label, "check.bin")
```

Python